

Resumo de Estudos — LDM

*Métricas de ML, fundamentos matemáticos,
Transformers e arquitetura LDM*

15 temas • 16 slides

Síntese das anotações manuscritas, com
folha original anexa em cada slide.

- 01 • Conceitos 02 • LDM substitui
- 03 • Feature Eng. 04 • Solução LDM
- 05 • Fundamentos 06 • Sigmoid
- 07 • Quadrantes 08 • AUROC / AUPRC
- 09 • Over/Underfit 10 • Transformer
- 11 • Conceitos T. 12 • Época/Batch
- 13 • Q, K, V 14 • Multi-head
- 15 • Arquitetura LDM

Precisão, Recall & Acurácia

Métrica • Precisão

Precision

Reflete a **qualidade das classificações** do modelo com sucesso. Dos itens que o modelo identificou, quantos eram verdadeiros.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Ex: de 100 transações sinalizadas, 60 eram fraudes e 40 transações legítimas perdidas.

$$\text{Precision} = 60 / (60+40) = 60\%$$

Métrica • Recall

Recall

Identificado em um **universo absoluto**: qual foi a **detecção dos elementos reais identificados**.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Ex: de 200 fraudes, o modelo encontrou 160 e deixou passar 40.

$$\text{Recall} = 160 / (160+40) = 80\%$$

Métrica • Acurácia

Accuracy

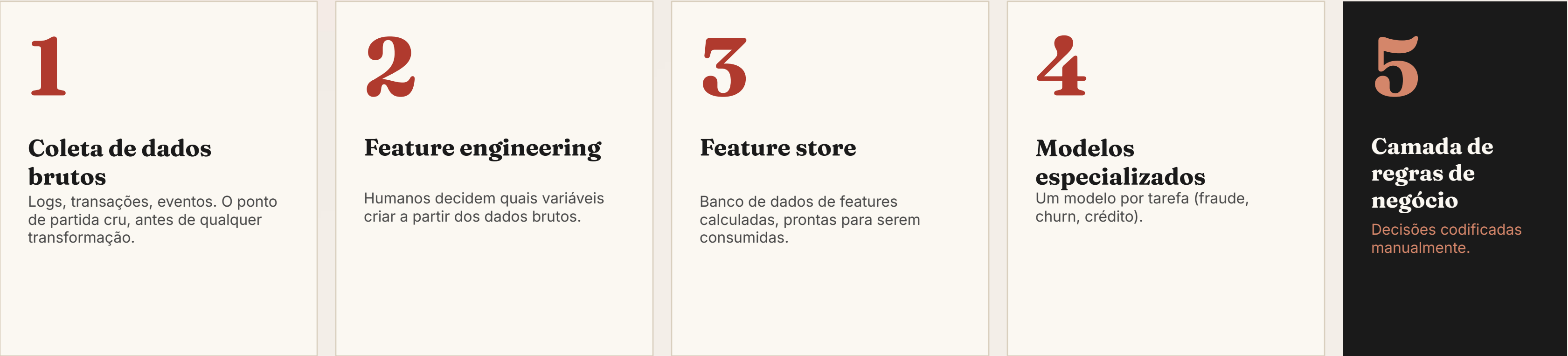
Equilíbrio entre Precision e Recall, principalmente quando a classe é rara.

⚠ Atenção em datasets desbalanceados — em fraude, a maioria das transações é legítima, então acurácia alta pode esconder modelo ruim.

Resumo: nem precision nem recall sozinhos — equilíbrio entre os dois.

A pipeline básica em 5 etapas

O LDM substitui o trabalho manual das 5 etapas abaixo, automatizando o que humanos decidem em cada uma.



Custo de manutenção: quando muda o mercado (PIX, por exemplo), todas as features precisam ser redesenhadas do zero — o que gera retrabalho.

As 5 limitações estruturais do pipeline clássico

O whitepaper da NeoSpace identifica 10 limitações estruturais do pipeline clássico. As 5 mais importantes para memorizar:

03 / 15

 [folha-03-feature-engineering.jpg](#)

Limitação 01 Rigidez temporal	Mudanças de mercado invalidam features. O modelo degrada e demora meses para ser corrigido.
Limitação 02 Mean tyranny	Médias apagam picos comportamentais críticos. 3 reclamações em 2 dias $\neq$ 3 reclamações em 60 dias — a média é igual.
Limitação 03 Knowledge imprinting	Bias humano embutido nas features. O modelo confirma hipóteses existentes em vez de descobrir padrões novos.
Limitação 04 Explosão de modelos	Uma empresa pode ter 80+ modelos em produção, cada um com feature engineering separado e equipe dedicada.
Limitação 05 Inexpressividade de texto	Feature stores não conseguem processar texto (chats, tickets, notas) de forma significativa.

Modelo treinado direto no event stream

Transformer treinado diretamente no event stream bruto — análogo ao que LLMs fazem com texto puro.

Pipeline clássica

Feature engineering manual

- Um modelo por tarefa
- Features agregadas (médias)
- Rígido a mudança de mercado
- Não processa texto

Custo: N equipes × N modelos

Solução LDM

LDM — modelo unificado

- Event stream bruto, bem-feature
- Sequência completa de eventos
- Aprende nuances efetivamente
- Texto é um conteúdo nativo

Custo: 1 modelo + 1 fine-tuning

Como o modelo toma a decisão

Conceito de Score

Um número entre 0 e 1 que representa a probabilidade estimada de pertencer a uma classe positiva.

*Ex: fraude / churn. O primeiro passo é calcular a soma ponderada das features com os pesos atribuídos no treinamento — esse cálculo chamamos de **Z**.*

Intuição

Cada feature contribui de acordo com seu peso. Quanto maior o peso, mais aquela feature pesa na decisão final do modelo.

Soma ponderada • Z

$$Z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

onde:

w = peso

x = feature

b = bias

O bias é o ponto de partida do modelo, sem ele a previsão de decisão sairia forçada pela origem; limitando o que o modelo pode aprender — com o bias, o modelo tem mais flexibilidade nos pesos.

Do Z bruto à probabilidade

Z pode ser qualquer número (positivo, negativo, grande, pequeno). A sigmoide “espreme” esse valor para ficar entre 0 e 1, tornando-o interpretável como probabilidade.

Fórmula

Score = 1 / (1 + e^{-z})

onde **e** = Euler (~ 2.718)

Quanto maior o Z, mais próximo de 1 (alta probabilidade da classe positiva). Quanto menor (mais negativo), mais próximo de 0.

Tabela de valores • sigmoide

Z	Cálculo	Score	Interpretação
-3	1/(1+e ³)	0.05	5% — extremamente negativo
-1	1/(1+e)	0.27	27% — provável negativo
0	1/(1+e ⁰)	0.50	50% — sem certeza
1	1/(1+e ⁻¹)	0.73	73% — provavelmente positivo
3	1/(1+e ⁻³)	0.95	95% — extremamente positivo

Os 4 quadrantes da classificação

Cruzando o que o modelo previu com a realidade, existem 4 resultados possíveis.

	MODELO DISSE: POSITIVO	MODELO DISSE: NEGATIVO
ERA: POSITIVO	TP ✓ True Positive (acerto) Ex: disse fraude, era fraude	FN ✗ Falso Negativo (erro) Ex: disse não fraude, era fraude Custo: prejuízo financeiro
ERA: NEGATIVO	FP ✗ Falso Positivo (alarme falso) Ex: disse fraude, era legítima Custo: cliente bloqueado	TN ✓ True Negative (acerto) Ex: disse legítima, era legítima

Tensão fundamental

Precision ↔ Recall

↑ Precision

Diminui FP (menos alarmes falsos), aumenta FN — perde fraudes reais.

↑ Recall

Pega todas as fraudes reais, mas quanto encontradas? Aumenta FP.

“Quem decide → o negócio.”

Métricas além do threshold único

Ambas medem a qualidade do modelo como um todo, independente do limiar escolhido.

Métrica 01

AUROC

Mede a capacidade do modelo de **ranquear** positivos acima de negativos, em qualquer limiar.

0.5 aleatório
1.0 perfeito

✓ *Bom para datasets balanceados*

Métrica 02

AUPRC

Equilíbrio entre Precision e Recall ao longo de todos os limiares.

Melhor para dataset desbalanceado — quando a classe rara (ex: fraude) tem 0.17% de ocorrência positiva.

✓ *Bom para dataset desbalanceado*

Métrica 03

F1-Score

Média harmônica entre Precision e Recall (fixados num limiar).

$$F1 = 2 \cdot (P \cdot R) / (P + R)$$

✓ *Bom para uma métrica de equilíbrio*

Os 3 regimes de aprendizado

Comparando o desempenho no treino e no teste, conseguimos diagnosticar a saúde do modelo.

Regime 01

Underfitting

Estudou pouco. Errou em tudo — no treino e no teste.

AUROC treino: 0.61
AUROC teste: 0.59

Regime 02

Bom modelo

Aprendeu padrões reais. Vai bem no treino e no teste — generaliza.

AUROC treino: 0.87
AUROC teste: 0.84

Regime 03

Overfitting

Decorou as respostas. Perfeito no treino, péssimo no teste.

AUROC treino: 0.99
AUROC teste: 0.71

Regra: se $AUROC_{treino} - AUROC_{teste} > 0.05 \Rightarrow \text{overfitting}$. Mecanismo anti-overfitting: parar quando o desempenho no teste para de melhorar.

Arquitetura que revolucionou NLP



Arquitetura de rede neural criada em 2017, revolucionando NLP — e a base por trás dos LLMs e LDMs para dados comportamentais.

A intuição-chave

*Um Transformer lê uma sequência de tokens e prevê quais tokens são mais relevantes para entender cada outro token. Isso é o mecanismo de **self-attention** — muito mais relevante que avaliar a “bombinha” fora ou um login normal de 30 dias atrás.*

No LDM

Cada cliente é um token. O Transformer aprende qual evento importa.

Self-attention

Cada token avalia quanto cada outro token “pesa” para si.

Os blocos elementares

Conceito 01

Token

Unidade mínima de entrada. Em texto: uma palavra. No LDM: um evento (transação, reclamação, login).

Conceito 02

Embedding • Vetor

Representação numérica (N dimensões) do token. Tokens semelhantes ficam próximos no espaço vetorial.

Conceito 03

Positional encoding

O Transformer não tem noção de ordem — somamos um vetor que codifica a posição do token.

Conceito 04 • o coração

Self-attention

Mecanismo que calcula, para cada token, o quanto os outros tokens da sequência pesam na sua representação. O cálculo final de saída de cada token recebe peso 0.07, equivalente.

Conceito 05

Multi-head attention

Múltiplas “cabeças” de atenção em paralelo, cada uma captura tipos de atenção diferentes (uma para padrões temporais, outra para relação de valor, outra para sequência).

Como o modelo aprende no tempo

Hiperparâmetro 01

Época

Uma passagem completa pelo dataset de treinamento.

O modelo ajusta os pesos a cada época. Mais épocas:

- **Bem ajustadas** = mais aprendizado
- **Excesso** = risco de overfitting

Hiperparâmetro 02

Batch

Subconjunto dos dados processado de uma vez para atualizar os pesos.

Batch maior

Treinamento mais rápido, mas mais memória.

Batch menor (ex: 512)

Modelo aprende com mais detalhes, mais ruído nos passos.

Hiperparâmetro 03

Patience

Critério de parada antecipada para evitar overfitting.

Definição: se depois de N épocas sem melhora no conjunto de validação, para o treinamento. Evita overfitting.

Ex: patience = 10

Learning rate: tamanho do passo ao ajustar os pesos. Muito alto = aprendizado instável • muito baixo = aprendizado lento.

Query, Key, Value

Usa três vetores derivados de cada token: a relação entre eles produz o peso de atenção.

Q • Query Pergunta O que cada token está “procurando” entre os outros eventos passados. Analogia: o que você digita na busca no Google.	K • Key Resposta O que cada token tem para oferecer. É o título que cada evento combina com o Key. Analogia: os títulos dos resultados.	V • Value Conteúdo O conteúdo real do evento, usado quando o Query combina com o Key. Analogia: o conteúdo da página.
---	---	---

Fórmula de atenção

Attention(Q, K, V) = softmax(Q · K^T / √d) · V

Q · K^T → produto interno mede a “match” entre Query e Key · √d → normaliza para estabilizar gradientes · softmax → distribui pesos somando 1

Single vs Multi-head

Variante 01

Single-head

- uma passagem de atenção
- captura um tipo de relação
- perde nuances sutis ou paralelas

Boa apenas para padrões mais simples — perde paralelismo.

Variante 02

Multi-head

- N passagens em paralelo
- cada cabeça aprende um tipo de relação
- concatena resultados temporais, semânticas, de papel
- captura relações em paralelo, ao mesmo tempo

Fórmula

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_N) \cdot W^o$$

onde $\text{head}_i = \text{Attention}(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V)$

Os 5 blocos do LDM

LDM da NeoBank possui 5 blocos distintos, cada um com um Transformer especializado.



Fluxo: evento bruto → embedding por campo → fusão local → contexto temporal → contexto entre clientes → latent state final.